

ANÁLISE DE TRÁFEGO E CONTAGEM DE OBJETOS COM **YOLOv8**



COMO UM CLIENTE QUALQUER, VEM- SE A QUESTÃO...

Como podemos monitorar e quantificar, de forma automatizada, o fluxo de carros em um ambiente de vídeo com precisão, velocidade e rastreamento contínuo?



MAS E A SOLUÇÃO? VISÃO COMPUTACIONAL

Implementei um pipeline robusto usando preferencialmente o YOLOv8 para detectar objetos, acompanhado da biblioteca SUPERVISION da ROBOFLOW para os complementos, como o rastreamento e as "zonas" de contagem.

Mas primeiro, o que é

YOLO ?



YOU ONLY LIVE ONCE É PASSADO...

Y (OU)

O (NLY)

L (OOK)

O (NCE)

é um algoritmo de detecção de objetos em tempo real baseado em redes neurais convolucionais. Diferentemente de outros métodos que dividem a imagem em várias regiões e as processam separadamente, o YOLO analisa toda a imagem uma **única** vez, estimando diretamente as coordenadas das caixas delimitadoras (bounding boxes) e as probabilidades de cada classe de objeto.

Como funciona?

O YOLO divide a imagem de entrada em uma grade (por exemplo, 13x13 ou 19x19 células). Para cada célula, a rede prediz várias caixas delimitadoras e a probabilidade de cada classe estar presente naquela região. O modelo é treinado end-to-end, o que significa que toda a rede é otimizada para maximizar a precisão das detecções. A velocidade é alcançada porque o processamento ocorre em uma única passada pela rede, diferentemente de métodos como R-CNN que requerem múltiplas passadas.

YOLO NO MULTIVERSO? AS INÚMERAS VERSÕES DO YOLO...

Para chegar aonde estamos, a Ultralytics passou por muitas versões para alcançar o que antes parecia inalcançável, Mais de 11 versões que estão oficialmente disponíveis até hoje, sendo as principais:

YOLOv8

- **Foco:** Detecção padrão otimizada
- **Vantagem:** Precisão superior e velocidade elevada
- **Casos de uso:** Detecção em cenários com classes pré-definidas
- **Características:** Suporta detecção, segmentação, pose estimation e classificação

YOLO-World

- **Foco:** Detecção com vocabulário aberto
- **Vantagem:** Detecção flexível por prompts de texto
- **Casos de uso:** Detectar objetos não pré-treinados usando descrições textuais
- **Características:** Zero-shot learning com texto, maior consumo de recursos

YOLOv11

- **Foco:** Eficiência e equilíbrio
- **Vantagem:** Arquitetura otimizada, menos parâmetros, treinamento mais rápido
- **Casos de uso:** Aplicações com restrições computacionais
- **Características:** Versão mais recente e refinada do pipeline YOLO

YOLOe

- **Foco:** Detecção universal com múltiplos prompts
- **Vantagem:** Zero-shot superior, 1.4× mais rápido que YOLO-World, menos recursos
- **Casos de uso:** Cenários com vocabulário aberto e restrições de performance
- **Características:** Suporta prompts de texto, visuais e vocabulário interno

YOLO -> PRINCIPAIS PARÂMETROS

conf (confiança): limiar mínimo de confiança para detectar um objeto. Valores mais altos (ex: 0.7) reduzem falsos positivos. **iou** (Intersection over Union): limiar para supressão de não-máximo, eliminando detecções duplicadas.

source: origem dos dados (imagem, vídeo, câmera, diretório).

device: dispositivo computacional ('cpu', '0', '1' para GPU). **save**: salvar anotações e resultados.

imgsz: tamanho da imagem de entrada (640 por padrão). **max_det**: número máximo de detecções por imagem.

augment: aplicar aumento de dados durante inferência. **persist**: manter rastreamento entre frames em vídeos.

ATENÇÃO! A DIFERENÇA ENTRE TRACKING E INFERÊNCIA PADRÃO

Inferência padrão detecta objetos em cada frame de forma independente, sem manter qualquer informação sobre frames anteriores. Cada detecção é isolada.

Tracking (rastreamento) mantém a identidade dos objetos ao longo do tempo. Quando você ativa o rastreamento, o YOLO não apenas detecta objetos em cada frame, mas também associa detecções de frames consecutivos ao mesmo objeto, atribuindo IDs únicos. Isso é essencial para aplicações como contagem de pessoas, análise de movimento ou seguimento de objetos específicos.

TESTE VOCÊ MESMO!

```
from ultralytics import YOLOE
import cv2

model = YOLOE("yoloe-lls-seg.pt")
names = ["nome do que quer
detectar"]
model.set_classes(names,
model.get_text_pe(names))
model.predict("suaimagem.jpg",
              show=True,
              save=True,)
```

YOLO E SUPERVISION

DETECÇÃO E
RASTREAMENTO



SIMPLES E
EFICIENTE



supervision

democratising computer vision • powered by **roboflow**

CONTAGEM EM ÁREA

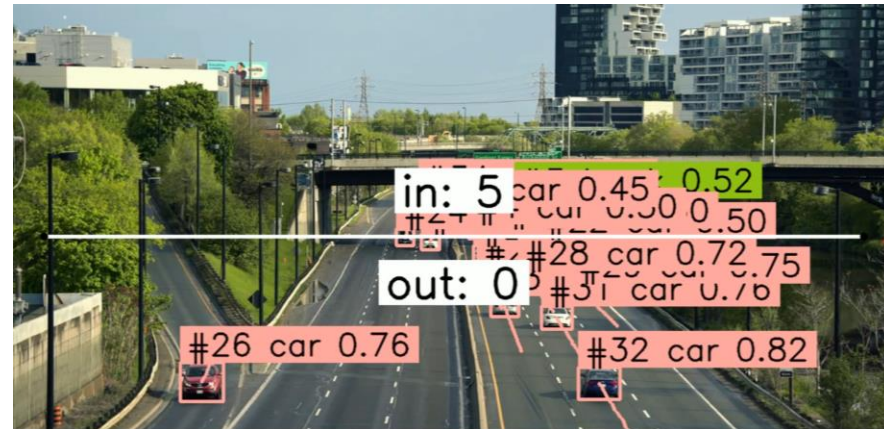
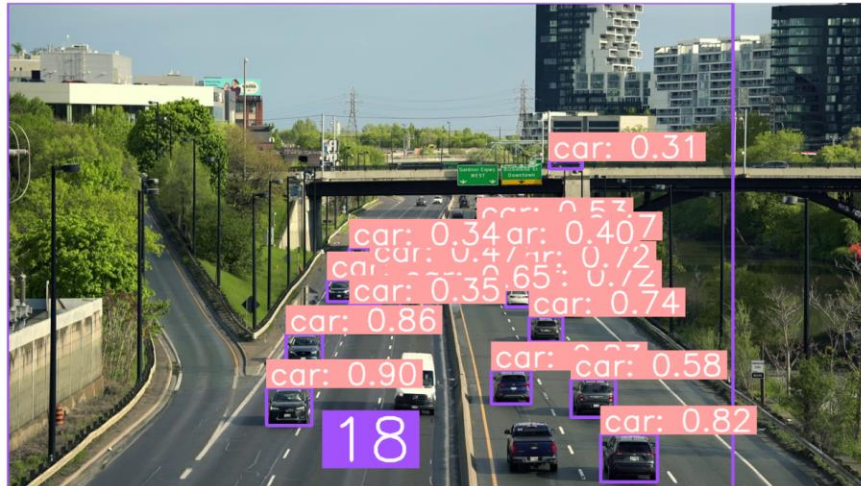
(barelyworkingasintended.ipynb)

https://colab.research.google.com/drive/18u94ig6sGRd_42WuNhmPxCB3jUXwmooV?usp=sharing

IN N' OUT

(trackcountREAL.ipynb)

<https://colab.research.google.com/drive/1e89bFEOlsFxj55pBIWL8HIkgTLQOTmut?usp=sharing>



TRACKCOUNTREAL.IPYNB -- CALLBACK

Esse sistema tem como importância a sua **função de callback**.

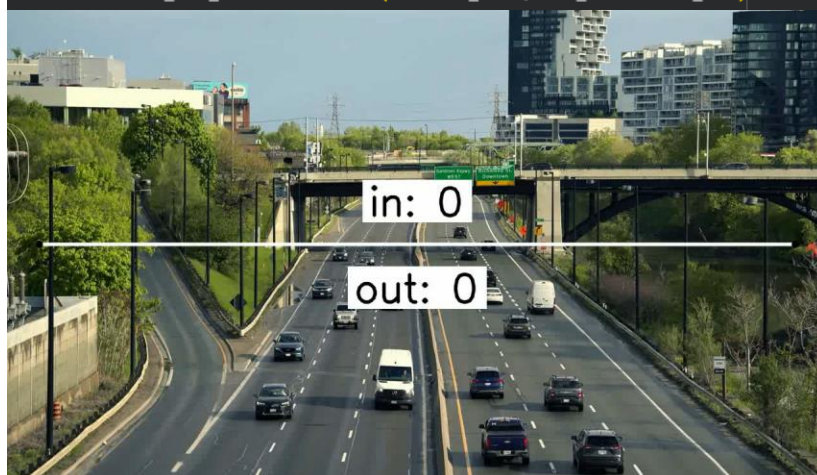
Começamos definindo o começo e o final da linha para cravarmos diretamente no **MEIO** (de acordo com a imagem informada), o que estabelece a **Região de Interesse (ROI)** e inicializa o objeto **sv.LineZone**.

Após o mencionado, partimos para a **função callback**, que é **executada para cada frame do vídeo** (garantindo o processamento em tempo real).

Desconsiderando já a detecção (YOLOv8) e o rastreamento de ID (ByteTrack), observamos o uso do **detections** (agora enriquecido com o **tracker_id**) sendo passado para o método **line_zone.trigger(detections)**. Esta chamada aciona o **gatilho de cruzamento**, que verifica se o **centroide** do objeto cruzou a linha, aumentando assim a contagem direcional.

```
def callback(frame: np.ndarray, index: int) -> np.ndarray:
    # model prediction on single frame and conversion to supervision Detections
    results = model(frame, verbose=False)[0]
    detections = sv.Detections.from_ultralytics(results)
    # only consider class id from selected_classes define above
    detections = detections[np.isin(detections.class_id, SELECTED_CLASS_IDS)]
    # tracking detections
    detections = byte_tracker.update_with_detections(detections)
    labels = [
        f"#{tracker_id} {model.model.names[class_id]} {confidence:0.2f}"
        for confidence, class_id, tracker_id
        in zip(detections.confidence, detections.class_id, detections.tracker_id)
    ]
    annotated_frame = frame.copy()
    annotated_frame = trace_annotator.annotate(
        scene=annotated_frame, detections=detections)
    annotated_frame = box_annotator.annotate(
        scene=annotated_frame, detections=detections)
    annotated_frame = label_annotator.annotate(
        scene=annotated_frame, detections=detections, labels=labels)

    # update line counter
    line_zone.trigger(detections)
    # return frame with box and line annotated result
    return line_zone_annotator.annotate(annotated_frame, line_counter=line_zone)
```



```
byte_tracker = sv.ByteTrack(
    track_activation_threshold=0.25,
    lost_track_buffer=30,
    minimum_matching_threshold=0.8,
    frame_rate=30,
    minimum_consecutive_frames=3)

byte_tracker.reset()

# create VideoInfo instance
video_info = sv.VideoInfo.from_video_path(SOURCE_VIDEO_PATH)

# create frame generator
generator = sv.get_video_frames_generator(SOURCE_VIDEO_PATH)

# create LineZone instance, it is previously called LineCounter class
line_zone = sv.LineZone(start=LINE_START, end=LINE_END)

# create instance of BoxAnnotator, LabelAnnotator, and TraceAnnotator
box_annotator = sv.BoxAnnotator(thickness=4)
label_annotator = sv.LabelAnnotator(text_thickness=2, text_scale=1.5, text_color=sv.Color.BLACK)
trace_annotator = sv.TraceAnnotator(thickness=4, trace_length=50)

# create LineZoneAnnotator instance, it is previously called LineCounterAnnotator class
line_zone_annotator = sv.LineZoneAnnotator(thickness=4, text_thickness=4, text_scale=2)
```

```
# settings
#LINE_START = sv.Point(0 + 50, 1500)
#LINE_END = sv.Point(3840 - 50, 1500)

MEIO_Y = 720 // 2 # Resultado: 360

LINE_START = sv.Point(x=0 + 50, y=MEIO_Y)
LINE_END = sv.Point(x=1280 - 50, y=MEIO_Y)

TARGET_VIDEO_PATH = f"{HOME}/result.mp4"
```


BARELYWORKINGASINTENDED.IPYNB -- POLYGONS E ZONAS DE INTERESSE

Mesmo sendo extremamente impreciso, foi-se usado a **PolygonZone** (do *framework* Supervision) para criar uma **grande zona retangular** (que visualmente se manifestou como um "quadrado") no vídeo visando testar e controlar a área de interesse. O objetivo principal era **quantificar a densidade e a ocupação veicular** dentro dessa área delimitada. Para garantir a relevância da contagem, um filtro rigoroso foi aplicado, considerando apenas as detecções da classe **car** (ou `class_id == 2`) que apresentassem um **nível de confiança igual ou superior a 20%**, garantindo que apenas objetos de alta probabilidade fossem considerados dentro dos limites do polígono.

```
polygons = [
    np.array([
        [0, 0],
        [1080 - 5, 0],
        [1080 - 5, 1300 - 5],
        [0, 1300 - 5]
    ], np.int32)

]

colors = sv.ColorPalette.DEFAULT

# Zonas
zones = [
    sv.PolygonZone(polygon=polygon)
    for polygon in polygons
]
```



```
# tracking detections
detections = byte_tracker.update_with_detections(detections)
labels = [
    f"#{tracker_id} {model.model.names[class_id]} {confidence:0.2f}"
    for confidence, class_id, tracker_id
    in zip(detections.confidence, detections.class_id, detections.tracker_id)
]
```

CONCLUSÕES?

Provando-se capaz de ser aplicável em qualquer circunstância, YOLO cumpre seu trabalho quase que precisamente detectando o que foi alimentado pelo COCO (dataset!!!).

Para aplicações logísticas levamos em consideração que *trackcount* é ideal para análise de fluxo, como esteiras de produtos, saídas de túneis e portarias de condomínios.

Barelyworking é essencial para um monitoramento de ocupação. O uso dos polígonos permite definir vagas ou áreas de estoques com precisão quando necessário. Em tempo real ou como imagem é extremamente útil para análise de densidade e controle de multidão.



REFERÊNCIAS BIBLIOGRÁFICAS

Ultralytics:

<https://docs.ultralytics.com/models/yolov8/>

<https://docs.ultralytics.com/datasets/detect/coco/>

<https://www.youtube.com/watch?v=vKcD44GkSF8>

https://www.youtube.com/watch?v=Ag2e-5_NpS0

<https://www.youtube.com/watch?v=uaEwlqXKnKo>

Roboflow:

<https://youtu.be/OS5ql9YBkfk?si=IVp0fYYuEdN6ZLQ7>

<https://youtu.be/wuZtUMEiKWY?si=vXO30hUJ3NBzEI5n>